

TUGATAI AI SAAS PLATFORM

DATABASE ERD DESIGN DOCUMENT

Prepared By

Md. Shahriar Hossain Jihad

Version

Version 1.0

Database Architecture

PostgreSQL + Prisma + pgvector

Date

May 2026

TABLE OF CONTENTS

1. Database Overview
2. Database Design Principles
3. Multi-Tenant Database Strategy
4. Branch Hierarchy Strategy
5. Core Database Modules
6. Entity Relationship Overview
7. Core Tables
8. Authentication & RBAC Tables
9. Tenant & Branch Tables
10. Omnichannel Inbox Tables
11. CRM Tables
12. Booking Tables
13. Pricing Engine Tables
14. AI & Knowledge Base Tables

15. Workflow Engine Tables
 16. Campaign Tables
 17. Analytics Tables
 18. Billing & Subscription Tables
 19. Notification Tables
 20. Audit & Activity Tables
 21. Database Relationships
 22. Indexing Strategy
 23. Vector Database Design
 24. Prisma ORM Strategy
 25. Future Scalability Strategy
 26. Database Best Practices
 27. Conclusion
-

1. DATABASE OVERVIEW

The Tugatai AI SaaS Platform uses PostgreSQL as the primary relational database system.

The database architecture is designed for:

- multi-tenant SaaS scalability
 - branch-level operational hierarchy
 - omnichannel messaging
 - CRM workflows
 - AI knowledge systems
 - workflow automation
 - analytics
 - realtime operations
 - future scalability
-

2. DATABASE DESIGN PRINCIPLES

The database follows:

- normalized relational design
 - tenant_id isolation
 - branch hierarchy isolation
 - modular table grouping
 - audit-friendly structure
 - scalability-first design
 - minimal redundancy
 - extensible schema strategy
-

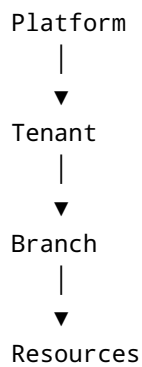
3. MULTI-TENANT DATABASE STRATEGY

Architecture Strategy

The platform uses:

- shared PostgreSQL database
 - tenant_id-based isolation
-

Tenant Isolation Flow



```
graph TD; Platform --> Tenant; Tenant --> Branch; Branch --> Resources;
```

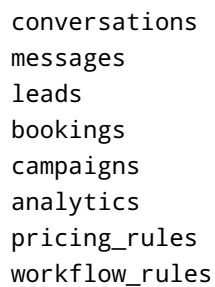
A vertical flow diagram showing the hierarchy of tenant isolation. It starts with 'Platform' at the top, followed by a downward arrow to 'Tenant', another downward arrow to 'Branch', and a final downward arrow to 'Resources'.

Tenant Isolation Rules

Every major table MUST contain:

- tenant_id

Examples:



```
conversations
messages
leads
bookings
campaigns
analytics
pricing_rules
workflow_rules
```

A list of database tables, each on a new line, enclosed in a rounded rectangle. The tables are: conversations, messages, leads, bookings, campaigns, analytics, pricing_rules, and workflow_rules.

4. BRANCH HIERARCHY STRATEGY

Branch Architecture

Each tenant can have:

- multiple branches
 - branch-specific users
 - branch-specific conversations
 - branch-specific CRM
 - branch-specific campaigns
 - branch-specific analytics
-

Branch Flow

```
Tenant
|
├── Doha Branch
├── Kampala Branch
├── Nairobi Branch
└── Ghana Branch
```

Branch Isolation Rules

Operational tables MUST include:

- branch_id

Examples:

```
conversations
leads
bookings
campaigns
notifications
```

5. CORE DATABASE MODULES

Authentication Module
Tenant Module
Branch Module
Inbox Module
CRM Module
Booking Module
Pricing Module
Workflow Module
Campaign Module
AI Module
Knowledge Base Module
Analytics Module
Billing Module
Notification Module
Audit Module

6. ENTITY RELATIONSHIP OVERVIEW

```
Tenant
├── Branch
│   ├── Users
│   ├── Conversations
│   ├── Leads
│   ├── Bookings
│   ├── Campaigns
│   └── Analytics
├── Subscription
├── Workflow Rules
├── Pricing Rules
└── Knowledge Base
```

7. CORE TABLES

Core Tables List

```
tenants
branches
users
roles
permissions
channels
conversations
messages
contacts
leads
crm_notes
bookings
pricing_rules
campaigns
workflow_rules
subscriptions
payments
knowledge_documents
ai_embeddings
analytics
notifications
audit_logs
activity_logs
```

8. AUTHENTICATION & RBAC TABLES

8.1 roles

Column	Type
id	UUID
name	VARCHAR
description	TEXT
created_at	TIMESTAMP

8.2 permissions

Column	Type
id	UUID
name	VARCHAR
module	VARCHAR
created_at	TIMESTAMP

8.3 role_permissions

Column	Type
role_id	UUID
permission_id	UUID

8.4 users

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
role_id	UUID
name	VARCHAR
email	VARCHAR
password_hash	TEXT
phone	VARCHAR
avatar	TEXT
status	ENUM
last_login	TIMESTAMP
created_at	TIMESTAMP

Column	Type
updated_at	TIMESTAMP

9. TENANT & BRANCH TABLES

9.1 tenants

Column	Type
id	UUID
name	VARCHAR
slug	VARCHAR
logo	TEXT
industry	VARCHAR
timezone	VARCHAR
status	ENUM
created_at	TIMESTAMP
updated_at	TIMESTAMP

9.2 branches

Column	Type
id	UUID
tenant_id	UUID
name	VARCHAR
country	VARCHAR
city	VARCHAR
address	TEXT
phone	VARCHAR
email	VARCHAR

Column	Type
manager_id	UUID
status	ENUM
created_at	TIMESTAMP
updated_at	TIMESTAMP

10. OMNICHANNEL INBOX TABLES

10.1 channels

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
type	ENUM
external_id	VARCHAR
access_token	TEXT
webhook_secret	TEXT
status	ENUM
created_at	TIMESTAMP

10.2 contacts

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
name	VARCHAR
phone	VARCHAR

Column	Type
email	VARCHAR
source	VARCHAR
metadata	JSONB
created_at	TIMESTAMP

10.3 conversations

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
contact_id	UUID
assigned_user_id	UUID
channel_id	UUID
status	ENUM
priority	ENUM
ai_enabled	BOOLEAN
last_message_at	TIMESTAMP
created_at	TIMESTAMP
updated_at	TIMESTAMP

10.4 messages

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
conversation_id	UUID

Column	Type
sender_type	ENUM
sender_id	UUID
message_type	ENUM
content	TEXT
media_url	TEXT
ai_generated	BOOLEAN
metadata	JSONB
created_at	TIMESTAMP

11. CRM TABLES

11.1 leads

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
contact_id	UUID
assigned_user_id	UUID
source	VARCHAR
stage	ENUM
score	INTEGER
summary	TEXT
ai_sentiment	VARCHAR
created_at	TIMESTAMP
updated_at	TIMESTAMP

11.2 crm_notes

Column	Type
id	UUID
tenant_id	UUID
lead_id	UUID
user_id	UUID
note	TEXT
created_at	TIMESTAMP

11.3 crm_activities

Column	Type
id	UUID
tenant_id	UUID
lead_id	UUID
type	VARCHAR
activity_data	JSONB
created_by	UUID
created_at	TIMESTAMP

12. BOOKING TABLES

12.1 bookings

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID

Column	Type
lead_id	UUID
booking_reference	VARCHAR
booking_type	VARCHAR
status	ENUM
pickup_location	TEXT
destination	TEXT
scheduled_time	TIMESTAMP
assigned_agent_id	UUID
total_amount	DECIMAL
created_at	TIMESTAMP
updated_at	TIMESTAMP

12.2 booking_items

Column	Type
id	UUID
booking_id	UUID
item_name	VARCHAR
quantity	INTEGER
weight	DECIMAL
dimensions	JSONB
unit_price	DECIMAL
total_price	DECIMAL

13. PRICING ENGINE TABLES

13.1 pricing_rules

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
service_type	VARCHAR
country	VARCHAR
pricing_type	VARCHAR
min_weight	DECIMAL
price_per_kg	DECIMAL
fixed_price	DECIMAL
surcharge	DECIMAL
metadata	JSONB
created_at	TIMESTAMP

13.2 pricing_logs

Column	Type
id	UUID
tenant_id	UUID
booking_id	UUID
pricing_snapshot	JSONB
calculated_amount	DECIMAL
created_at	TIMESTAMP

14. AI & KNOWLEDGE BASE TABLES

14.1 knowledge_documents

Column	Type
id	UUID
tenant_id	UUID
uploaded_by	UUID
title	VARCHAR
file_name	VARCHAR
file_type	VARCHAR
file_path	TEXT
processing_status	ENUM
created_at	TIMESTAMP

14.2 knowledge_chunks

Column	Type
id	UUID
document_id	UUID
chunk_text	TEXT
metadata	JSONB
created_at	TIMESTAMP

14.3 ai_embeddings

Column	Type
id	UUID
tenant_id	UUID

Column	Type
chunk_id	UUID
embedding	VECTOR
created_at	TIMESTAMP

14.4 ai_memory

Column	Type
id	UUID
tenant_id	UUID
conversation_id	UUID
memory_text	TEXT
embedding	VECTOR
created_at	TIMESTAMP

15. WORKFLOW ENGINE TABLES

15.1 workflow_rules

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
name	VARCHAR
trigger_type	VARCHAR
conditions	JSONB
actions	JSONB
is_active	BOOLEAN
created_at	TIMESTAMP

15.2 workflow_executions

Column	Type
id	UUID
workflow_id	UUID
execution_status	ENUM
execution_logs	JSONB
started_at	TIMESTAMP
completed_at	TIMESTAMP

16. CAMPAIGN TABLES

16.1 campaigns

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
created_by	UUID
name	VARCHAR
campaign_type	VARCHAR
status	ENUM
scheduled_at	TIMESTAMP
created_at	TIMESTAMP

16.2 campaign_recipients

Column	Type
id	UUID
campaign_id	UUID
contact_id	UUID
delivery_status	ENUM
delivered_at	TIMESTAMP

17. ANALYTICS TABLES

17.1 analytics

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
metric_name	VARCHAR
metric_value	DECIMAL
metadata	JSONB
created_at	TIMESTAMP

18. BILLING & SUBSCRIPTION TABLES

18.1 subscriptions

Column	Type
id	UUID
tenant_id	UUID

Column	Type
stripe_subscription_id	VARCHAR
plan_name	VARCHAR
status	ENUM
started_at	TIMESTAMP
expires_at	TIMESTAMP

18.2 payments

Column	Type
id	UUID
tenant_id	UUID
subscription_id	UUID
stripe_payment_id	VARCHAR
amount	DECIMAL
status	ENUM
paid_at	TIMESTAMP

19. NOTIFICATION TABLES

19.1 notifications

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
user_id	UUID
type	VARCHAR
title	VARCHAR

Column	Type
message	TEXT
is_read	BOOLEAN
created_at	TIMESTAMP

20. AUDIT & ACTIVITY TABLES

20.1 audit_logs

Column	Type
id	UUID
tenant_id	UUID
user_id	UUID
action	VARCHAR
module	VARCHAR
metadata	JSONB
ip_address	VARCHAR
created_at	TIMESTAMP

20.2 activity_logs

Column	Type
id	UUID
tenant_id	UUID
branch_id	UUID
user_id	UUID
activity_type	VARCHAR
activity_data	JSONB
created_at	TIMESTAMP

21. DATABASE RELATIONSHIPS

Primary Relationships

```
Tenant → Branches
Branch → Users
Branch → Conversations
Conversation → Messages
Contact → Leads
Lead → Bookings
Booking → Booking Items
Tenant → Pricing Rules
Tenant → Workflow Rules
Tenant → Campaigns
Tenant → Knowledge Base
```

Relationship Types

Relationship	Type
Tenant → Branch	One-to-Many
Branch → User	One-to-Many
Conversation → Messages	One-to-Many
Lead → Notes	One-to-Many
Booking → Items	One-to-Many
Role → Permissions	Many-to-Many

22. INDEXING STRATEGY

Important Indexes

```
INDEX tenant_id
INDEX branch_id
INDEX conversation_id
INDEX contact_id
```

```
INDEX lead_id
INDEX booking_reference
INDEX created_at
INDEX last_message_at
```

Full-Text Search

PostgreSQL full-text search should be used for:

- conversation search
- CRM search
- KB search

23. VECTOR DATABASE DESIGN

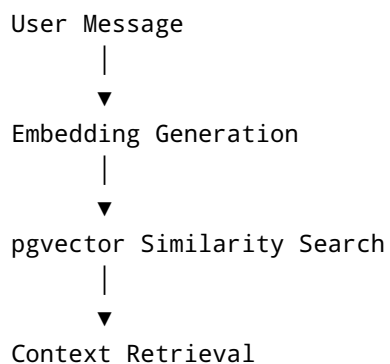
pgvector Strategy

The platform uses pgvector inside PostgreSQL.

Vector Responsibilities

- semantic KB search
- AI memory retrieval
- similarity search
- contextual retrieval

Vector Search Flow



24. PRISMA ORM STRATEGY

Prisma Responsibilities

Prisma handles:

- schema management
- migrations
- database access
- type-safe queries
- relations

Prisma Best Practices

- modular schema organization
- repository abstraction
- optimized queries
- scoped tenant queries

25. FUTURE SCALABILITY STRATEGY

Future Scaling Goals

The database architecture supports:

- future microservice extraction
- read replicas
- distributed workers
- analytics scaling
- AI scaling

Future Extraction Candidates

AI Service
Campaign Engine
Analytics Engine
Workflow Engine
Notification Service

26. DATABASE BEST PRACTICES

Required Best Practices

- use UUID primary keys
- use timestamps everywhere
- maintain audit logs
- use JSONB for flexible metadata
- avoid hard deletes
- use proper indexing
- maintain tenant isolation
- maintain branch isolation

Soft Delete Strategy

Recommended fields:

```
is_deleted  
deleted_at
```

27. CONCLUSION

The Tugatai AI SaaS database architecture is designed as a scalable multi-tenant relational system supporting:

- omnichannel communication
- AI workflows
- CRM
- campaigns
- workflow automation
- booking systems
- analytics
- SaaS scalability
- branch-based operations

The schema prioritizes:

- scalability
- maintainability
- extensibility

- performance
- tenant isolation
- operational efficiency
- future microservice readiness